

# Documentazione PrenotazioneService

## Scopo della classe

---

PrenotazioneService gestisce la logica di business relativa alla creazione e gestione delle prenotazioni per i campi sportivi (entità Prenotazione). Estende AbstractService<Prenotazione, PrenotazioneDto>, ereditando le operazioni CRUD generiche e aggiungendo la logica specifica per effettuare una prenotazione, verificare l'utente e la disponibilità, salvare i dati su database e inviare la relativa e-mail di conferma in formato HTML.

## Dipendenze

- **PrenotazioneMapper**: converte le entità Prenotazione nei rispettivi DTO e viceversa.
- **PrenotazioneRepository**: comunica con la base dati per persistere e recuperare le informazioni sulle prenotazioni.
- **UtenteRepository**: consente di verificare l'esistenza degli utenti registrati nel sistema prima di associare la prenotazione.
- **EmailService**: gestisce l'invio delle e-mail di notifica e conferma all'utente.
- **DisponibilitaCampoRepository**: permette di verificare ed estrarre la disponibilità specifica del campo selezionato per la prenotazione.

## Costruttore

---

Riceve tramite Dependency Injection il repository generico, il converter, il mapper specifico e i repository/servizi dedicati (PrenotazioneMapper, PrenotazioneRepository, UtenteRepository, EmailService e DisponibilitaCampoRepository). Richiama super(repository, converter) per inizializzare la superclasse e memorizza le dipendenze nei campi della classe.

## Metodi

---

### effetuaPrenotazione(PrenotazioneDto prenotazioneDto, Integer utenteld)

Gestisce l'intero ciclo di creazione e registrazione di una nuova prenotazione. Il flusso operativo si articola nelle seguenti fasi:

1. **Verifica Utente:** ricerca l'utente tramite `utenteRepository.findById(utenteld)`. Se l'utente non viene trovato, solleva un'eccezione generale (`Exception("Utente non trovato")`).
2. **Mappatura e Associazione:** converte il `PrenotazioneDto` nell'entità `Prenotazione` tramite `prenotazioneMapper.toEntity(prenotazioneDto)` e vi associa l'utente recuperato.
3. **Gestione Data Prenotazione:** controlla se la data/ora della prenotazione è valorizzata; in caso contrario, la imposta automaticamente al timestamp corrente (`LocalDateTime.now()`).
4. **Validazione Disponibilità Campo:** se nel DTO è presente un riferimento alla disponibilità del campo, interroga `disponibilitaCampoRepository.findById(...)` per verificarne l'esistenza. Se la disponibilità non viene trovata, solleva un'eccezione (`Exception("Disponibilità non trovata")`).
5. **Salvataggio su DB:** salva l'entità `Prenotazione` sul database tramite `prenotazioneRepository.save(prenotazione)`.
6. **Invio Email di Conferma:** estrae i dettagli (nome utente, nome del campo, data, ora e costo totale) e costruisce un template HTML formattato. Spedisce la mail all'indirizzo dell'utente mediante `emailService.sendEmail(...)`.
7. **Restituzione DTO:** converte l'entità salvata nel corrispondente `PrenotazioneDto` mediante `prenotazioneMapper.toDTO(saved)` e la restituisce al chiamante.

## Flusso di esecuzione

---

Controller → PrenotazioneService → UtenteRepository / DisponibilitaCampoRepository / PrenotazioneRepository → Database

PrenotazioneService → EmailService → Invio Email HTML all'utente

PrenotazioneSaved → PrenotazioneMapper → PrenotazioneDto → Controller → JSON

## Vantaggi

---

Separazione delle responsabilità, validazione preventiva dell'utente e della disponibilità del campo, automazione dell'impostazione della data di prenotazione, notifica immediata via e-mail HTML all'utente, conversione Entity/DTO centralizzata e sicura, codice isolato e facilmente manutenibile.